D Y PATIL INTERNATIONAL UNIVERSITY

School of Computer Science, Engineering & Applications

Sector 29, Pradhikaran, Akurdi, Pune – 411044

LAB MANUAL

Python Programming

Course Code: CSE1005P

Academic Year: 2026–27

B.Tech – Computer Science & Engineering | Semester I

Student Name	
Roll Number	
Division / Batch	
Faculty Name	

Index

Sr. No.	Experiment Title	Date	Marks	Sign
1	Implementation of Looping Constructs for Series Generation and Number Manipulation			
2	Implementation of Conditional Statements for Decision-Making Problems			
3	Design and Implementation of User-Defined Functions and Lambda Expressions			
4	Implementation of Python Data Structures Using Lists, Tuples, Sets, and Dictionaries			
5	Pattern Matching and Validation Using Regular Expressions in Python			
6	Design and Implementation of Object-Oriented Programming Concepts in Python			
7	Implementation of File Handling and Exception Handling Mechanisms in Python			
8	Data Analysis and Visualization Using NumPy, Pandas, and Matplotlib Libraries			

Course Information

Course Details	
Course Name	Python Programming
Course Code	CSE1005P
Credits	1 (L-T-P-C: 0-0-2-1)
Prerequisites	Basic knowledge of computer fundamentals
Academic Year	2026–27
Department	School of Computer Science, Engineering & Applications

Course Objectives

- Develop practical programming skills using Python constructs, control structures, and functions.
- Apply Python data structures and regular expressions for solving computational problems.
- Implement object-oriented programming, file handling, and exception handling concepts using Python.
- Use Python libraries for data analysis and visualization.

Course Outcomes

Course Outcomes (COs)	
CO1	Implement Python programs using looping constructs, conditional statements, and functions.
CO2	Apply Python data structures and regular expressions to solve computational problems.
CO3	Design and implement object-oriented programs using Python.
CO4	Implement file handling and exception handling mechanisms in Python applications.
CO5	Perform data analysis and visualization using NumPy, Pandas, and Matplotlib libraries.

Evaluation Scheme

Marks Distribution	
Lab Manual	10 marks
Lab Assessment	10 marks
Internal Viva-voce	30 marks
End-term Practical	50 marks
Total	100 marks

General Instructions

- Students must bring the lab manual to every lab session.
- Programs must be written neatly and executed before submission.
- Do not copy programs from peers; plagiarism will result in zero marks.
- Document the output and write your own conclusion for each experiment.
- Maintain the manual in a clean and organized manner throughout the semester.
- Reach the lab 5 minutes before the scheduled time.

Experiment 1

Experiment No.	1
Title	Implementation of Looping Constructs for Series Generation and Number Manipulation
Aim	To implement various looping constructs in Python for generating series and performing number manipulation.
Objectives	Understand for and while loops, use range(), implement series generation, apply loop control statements.
Date	
Marks	
Faculty Signature	

1. Theory

1.1 Introduction to Iteration

A program normally executes statements one after another in the order in which they are written. This is called sequential flow. Many problems, however, require the same set of statements to be executed repeatedly: printing the first 100 natural numbers, adding the marks of 60 students, or checking every number in a range for primality. Writing the same statement 100 times is impractical and inflexible, because the number of repetitions is often known only at run time (for example, when it is entered by the user). Iteration, implemented through loops, solves this problem by allowing a block of code to be executed repeatedly under the control of a condition or a sequence.

Every loop, regardless of its type, contains three logical components:

- **Initialization** – setting the starting value of the variable that controls the loop (e.g. count = 1).
- **Condition (Test)** – a Boolean expression that decides whether the loop body should execute once more (e.g. count <= 10).
- **Update** – a statement that changes the control variable so that the condition eventually becomes False (e.g. count += 1). If the update is missing or incorrect, the loop never terminates.

Each single execution of the loop body is called an iteration or pass. Python provides two looping constructs: the for loop and the while loop, supported by the loop control statements break, continue and pass, and by an optional else clause.

1.2 The for Loop

The Python for loop is a definite, iterator-based loop. Unlike the counter-based for loop of C or Java, it does not require an explicit initialization, test and increment. Instead, it walks through the items of an iterable object one at a time, assigning each item to the loop variable and then executing the loop body. The loop ends automatically when the items are exhausted.

Internally, Python calls iter() on the sequence to obtain an iterator, then repeatedly calls next() on it. When no items remain, the iterator raises a StopIteration signal, which the for statement handles silently to end the loop.

```
for ch in "PYTHON":
    print(ch, end=" ")       # Output: P Y T H O N


for fruit in ["apple", "mango", "banana"]:
    print(fruit.upper())     # APPLE  MANGO  BANANA
```

The objects that can follow the keyword in include strings (character by character), lists, tuples, sets, dictionaries (keys by default), range objects and file objects (line by line). The end=" " argument of print() replaces the default newline with a space, so the output stays on one line – a technique used throughout series-generation programs.

1.3 The range() Function

The built-in function range() generates an arithmetic progression of integers and is the most common companion of the for loop. Its general form is range(start, stop, step). The start value is included, the stop value is always excluded, and step is the difference between consecutive values (default 1). If only one argument is given, it is treated as stop and start defaults to 0.

In Python 3, range() does not return a list. It returns an immutable range object, a lazy sequence that computes each value only when it is requested. Consequently, range(10**9) occupies the same small amount of memory as range(10). To view all values at once, it can be converted using list(range(...)).

Call	Values Generated	Remarks
range(5)	0 1 2 3 4	start defaults to 0, stop 5 is excluded
range(2, 7)	2 3 4 5 6	start included, stop excluded
range(1, 10, 2)	1 3 5 7 9	step of 2 – odd numbers
range(10, 0, -3)	10 7 4 1	negative step counts downwards
range(5, 1)	(empty)	start ≥ stop with a positive step: loop body never runs

```
print(list(range(1, 10, 2)))   # [1, 3, 5, 7, 9]
print(type(range(5)))          # <class 'range'>
for i in range(5, 0, -1):
    print(i, end=" ")          # 5 4 3 2 1
```

1.4 The while Loop

The while loop is an indefinite, condition-controlled loop. It repeats its body as long as a given condition remains True. Because the condition is tested *before* each iteration, it is called an entry-controlled loop; if the condition is False at the very beginning, the body is not executed even once. The while loop is preferred when the number of iterations cannot be determined in advance – for example, extracting digits until a number becomes zero, or accepting input until the user enters a valid value.

```
count = 1                # initialization
while count <= 5:        # condition
    print(count, end=" ")
    count += 1           # update

# Output: 1 2 3 4 5
```

Python has no built-in do-while (exit-controlled) loop. The same behaviour – executing the body at least once – is obtained with while True combined with break:

```
while True:
    n = int(input("Enter a positive number: "))
    if n > 0:
        break            # leave the loop once input is valid
print("You entered", n)
```

1.5 Comparison of for and while Loops

Criterion	for Loop	while Loop
Type	Definite (iteration count known)	Indefinite (count may be unknown)
Controlled by	Items of an iterable / range	A Boolean condition

Criterion	for Loop	while Loop
Initialization and update	Handled automatically by the iterator	Must be written explicitly by the programmer
Risk of infinite loop	Very low	High if the update is forgotten
Typical use	Traversing sequences, fixed-count series	Digit processing, input validation, menus
Example	for i in range(10):	while num > 0:

Any for loop can be rewritten as a while loop and vice versa, but choosing the natural one makes the program shorter, clearer and less error-prone.

1.6 Loop Control Statements

Loop control statements change the normal sequence of execution inside a loop.

- **break** – immediately terminates the innermost enclosing loop. Control passes to the first statement after the loop. It is used when the purpose of the loop is already achieved, such as finding a divisor while testing primality.
- **continue** – skips the remaining statements of the current iteration and jumps to the next iteration. In Program 3, continue skips the numbers 0 and 1, which are not prime.
- **pass** – a null statement that does nothing. It is a placeholder used where Python syntax requires a statement but no action is needed yet (e.g. an empty loop body or a function still to be written).

```
for i in range(1, 10):
    if i == 3:
        continue          # skip 3
    if i == 7:
        break             # stop completely at 7
    print(i, end=" ")     # Output: 1 2 4 5 6


for i in range(5):
    pass                  # body to be written later
```

1.7 The else Clause with Loops

A distinctive feature of Python is that both for and while loops may have an else block. The else block executes only when the loop terminates normally, i.e. when the sequence is exhausted or the condition

becomes False. If the loop is exited through break, the else block is skipped. This makes the construct ideal for search problems: the else part represents the case "searched everything, found nothing".

Program 3 uses exactly this idea. The inner for loop searches for a divisor of num. If a divisor is found, break ends the search and the else is skipped; if no divisor exists, the loop finishes normally and the else block prints the number as prime.

```
num = 29
for i in range(2, int(num ** 0.5) + 1):
    if num % i == 0:
        print(num, "is not prime")
        break
else:
    print(num, "is prime")      # executes: no break occurred
```

1.8 Nested Loops

A loop placed inside the body of another loop is called a nested loop. For every single iteration of the outer loop, the inner loop completes all of its iterations. If the outer loop runs *m* times and the inner loop runs *n* times, the inner body executes $m \times n$ times. Nested loops are used for patterns, tables, matrices and for checking each number of a range for primality (Program 3). Note that break inside the inner loop terminates only the inner loop, not the outer one.

```
for row in range(1, 5):         # outer loop: rows
    for col in range(row):      # inner loop: columns
        print("*", end=" ")
    print()                     # new line after each row
# *
# * *
# * * *
# * * * *
```

1.9 Infinite Loops

An infinite loop is a loop whose terminating condition never becomes False. Accidental infinite loops are usually caused by forgetting the update statement, updating the wrong variable, or writing a condition that is always True (e.g. while n != 10 when n increases in steps of 3). Infinite loops are sometimes intentional – servers, games and menu-driven programs run while True: and leave the loop using break. A running infinite loop can be stopped from the keyboard with **Ctrl + C**, which raises KeyboardInterrupt.

```
i = 1
while i <= 5:
    print(i)        # BUG: i is never incremented -> infinite loop
```

1.10 Logic Behind the Programs

a) Fibonacci Series

The Fibonacci series is defined by the recurrence relation $F(0) = 0$, $F(1) = 1$ and $F(n) = F(n-1) + F(n-2)$ for $n \geq 2$, giving 0, 1, 1, 2, 3, 5, 8, 13, 21, … Only the last two terms are needed to compute the next one, so two variables a and b are sufficient. The statement a, b = b, a + b uses tuple packing and unpacking: the right-hand side is evaluated completely first (using the old values), and only then are the results assigned. This removes the need for a temporary variable. The loop runs *n* times, so the time complexity is O(n).

b) Sum of Digits (Digit Extraction)

Two integer operators form the basis of all digit-manipulation programs: num % 10 (modulus) gives the last digit of a number, and num // 10 (floor division) removes the last digit. Repeating these two steps until the number becomes 0 visits every digit exactly once. Since the number of digits is not known beforehand, a while loop is the natural choice. The trace for num = 472 is shown below.

Iteration	num (before)	digit = num % 10	total	num // 10
1	472	2	2	47
2	47	7	9	4
3	4	4	13	0
–	0	Condition num > 0 is False; loop ends	13	–

The same technique solves many related problems: counting digits, reversing a number (rev = rev * 10 + digit), checking a palindrome number, and checking an Armstrong number (sum of each digit raised to the power of the number of digits equals the number, e.g. $153 = 1^3 + 5^3 + 3^3$).

c) Prime Numbers in a Range

A prime number is a natural number greater than 1 that has exactly two divisors: 1 and itself. To test *n*, it is sufficient to check divisors from 2 up to $\sqrt{n}$. The reason is that if $n = a \times b$ with $a \leq b$, then a cannot exceed $\sqrt{n}$; therefore, if no divisor is found up to $\sqrt{n}$, none exists beyond it. For n = 97 this reduces the work from 95 trial divisions to only 8. In code, the limit is written as int(num ** 0.5) + 1

because range() excludes its stop value. The outer loop visits each number in the range and the inner loop performs the divisor test, giving an overall complexity of roughly O(n√n).

1.11 Common Errors and Good Practices

- **Indentation errors** – the loop body must be indented consistently (4 spaces is the convention); a missing colon after the loop header causes a SyntaxError.
- **Off-by-one errors** – forgetting that range() excludes the stop value, e.g. using range(1, n) when 1 to n inclusive is required (range(1, n + 1)).
- **Modifying the loop variable** inside a for loop does not affect the next iteration, because the variable is reassigned from the iterator each time.
- **Modifying a list while iterating over it** can skip elements; iterate over a copy (for x in lst[:]) instead.

Key Points: for is used when iterating over a known sequence; while is used when repetition depends on a condition. range(start, stop, step) excludes stop. break exits the loop, continue skips to the next iteration, pass does nothing. A loop's else block runs only if the loop was not terminated by break.

Programs

Program 1: Fibonacci Series using for loop

```
n = int(input("Enter number of terms: "))
a, b = 0, 1
print("Fibonacci Series:")
for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

Program 2: Sum of digits and reverse of a number using while loop

```
num = int(input("Enter a number: "))
total = 0
reverse=0
while num > 0:
    digit = num % 10
    total += digit
reverse=reverse*10+digit
    num //= 10
print("Sum of digits:", total)
print("Reverse of a number:", reverse)
```

Program 3: Prime numbers in a range

```
start = int(input("Start: "))
end = int(input("End: "))
print("Prime numbers:")
for num in range(start, end + 1):
    if num < 2:
        continue
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            break
    else:
        print(num, end=" ")
```

Practice Questions:

1. Sum of Natural Numbers using for loop
2. Factorial of a number using a loop
3. Check whether the number is a Armstrong number or not using loop

Viva Questions

1. What is the difference between for and while loop?
2. What does the range() function return?
3. Explain the use of break and continue.
4. What is an infinite loop?
5. What is a nested loop?

Assessment	
Timely Submission (5)	
Code Correctness (5)	
Viva Voce (10)	
Total Marks (20)	

Experiment 2

Experiment No.	2
Title	Implementation of Conditional Statements for Decision-Making Problems
Aim	To implement conditional statements (if, if-else, if-elif-else) to solve decision-making problems in Python.
Objectives	Apply if, if-else, elif ladders; solve real-world decision problems; use nested conditionals.
Date	
Marks	
Faculty Signature	

2. Theory

2.1 Decision Making and Flow of Control

The order in which statements are executed is called the flow of control. Every program is built from three basic control structures: sequence (statements executed one after another), selection (choosing between alternative paths) and iteration (repetition). Conditional statements implement selection. They allow a program to evaluate a condition and execute different blocks of code depending on whether the condition is True or False, which is how a program "makes decisions" – assigning a grade, validating an operator, or checking for division by zero.

2.2 Conditions and Boolean Expressions

A condition is any expression that evaluates to one of the two Boolean values True or False. Conditions are formed using relational (comparison) operators and combined using logical operators.

Operator	Meaning	Example (a = 10, b = 20)	Result
==	Equal to	a == b	False
!=	Not equal to	a != b	True
< / >	Less than / Greater than	a < b	True
<= / >=	Less / greater than or equal to	a >= 10	True
and	True if both operands are True	a > 5 and b > 25	False

Operator	Meaning	Example (a = 10, b = 20)	Result
or	True if at least one operand is True	a > 5 or b > 25	True
not	Reverses the Boolean value	not (a > 5)	False

Short-circuit evaluation: Python stops evaluating a logical expression as soon as its result is known. In x != 0 and y / x > 2, if x is 0 the second part is never evaluated, which prevents a division-by-zero error. Python also supports chained comparisons such as 0 <= marks <= 100, which is equivalent to 0 <= marks and marks <= 100.

Truthiness: In Python, any object can be used as a condition. The values False, None, 0, 0.0, the empty string "" and empty collections ([], (), {}, set()) are treated as False (falsy); every other value is truthy. Hence if name: is a concise way of checking that a string is not empty.

2.3 Blocks and Indentation

Python does not use braces { } to group statements. A block begins after a colon : and consists of all the following lines that are indented by the same amount (conventionally 4 spaces). The block ends when the indentation returns to the previous level. Incorrect or inconsistent indentation causes an IndentationError, or – worse – a logical error where a statement silently belongs to the wrong block.

2.4 The if Statement

The simple if statement executes its block only when the condition is True; when the condition is False the block is skipped and execution continues with the next statement. It represents a one-way decision. An if statement does not require an else part.

```
age = 20
if age >= 18:
    print("Eligible to vote")
print("Program continues")
```

2.5 The if-else Statement

The if-else statement is a two-way decision. Exactly one of the two blocks is always executed: the if block when the condition is True, the else block otherwise.

```
num = int(input("Enter a number: "))
if num % 2 == 0:
    print(num, "is even")
```

```
else:
    print(num, "is odd")
```

2.6 The if-elif-else Ladder

When there are more than two alternatives, the elif (short for "else if") keyword is used to build a multi-way decision ladder. Conditions are evaluated from top to bottom. As soon as one condition is True, its block is executed and all remaining conditions are skipped. The final else is optional and acts as the default case when none of the conditions is True. An if may be followed by any number of elif clauses.

Because only the first true condition is executed, the order of conditions matters. In the Grade Calculator (Program 1), the conditions start from the highest threshold (marks >= 90) and move downwards. If the check marks >= 40 were placed first, a student scoring 95 would satisfy it and wrongly receive grade D. Arranging thresholds in descending order also means each elif implicitly knows that all earlier conditions were False, so elif marks >= 75 actually represents the range 75–89.

Marks Range	Condition that becomes True first	Grade
90 – 100	marks >= 90	O (Outstanding)
75 – 89	marks >= 75	A (Excellent)
60 – 74	marks >= 60	B (Good)
50 – 59	marks >= 50	C (Average)
40 – 49	marks >= 40	D (Pass)
Below 40	none – else executes	F (Fail)

2.7 Nested Conditional Statements

An if statement placed inside another if, elif or else block is called a nested if. The inner condition is tested only when the outer condition is True. In the Simple Calculator (Program 2), the outer ladder selects the operation, and a nested if b != 0 inside the division branch protects against division by zero. Nesting should be kept shallow; deeply nested code is hard to read and can often be simplified using logical operators.

```
year = int(input("Enter year: "))
if year % 4 == 0:
    if year % 100 == 0:
        if year % 400 == 0:
```

```
        print("Leap year")
    else:
        print("Not a leap year")
else:
    print("Leap year")
else:
    print("Not a leap year")

# Equivalent single condition:
# if (year % 4 == 0 and year % 100 != 0) or year % 400 == 0:
```

2.8 Conditional Expression (Ternary Operator)

Python provides a one-line form of if-else called the conditional expression, commonly known as the ternary operator. Its syntax is value_if_true if condition else value_if_false. It is an *expression* (it produces a value), so it can be used inside assignments, function calls and f-strings. It should be used only for simple two-way choices.

```
marks = 55
result = "Pass" if marks >= 40 else "Fail"
print(result)                          # Pass
print("Even" if 7 % 2 == 0 else "Odd")        # Odd
```

2.9 The match-case Statement (Python 3.10+)

From Python 3.10 onwards, the match-case statement provides structural pattern matching, which is a cleaner alternative to long elif ladders that compare one variable against several constant values. The wildcard pattern _ acts as the default case. The calculator operator selection could be written as follows:

```
match op:
    case "+":
        print(a + b)
    case "-":
        print(a - b)
    case "*":
        print(a * b)
    case "/":
        print(a / b if b != 0 else "Error: Division by zero!")
```

```
    case _:
        print("Invalid operator")
```

2.10 The == Operator versus the is Operator

The == operator tests equality of values: whether two objects contain the same data. The is operator tests identity: whether two names refer to the very same object in memory (i.e. whether their id() values are equal). Two separately created lists with identical contents are equal but not identical. The is operator should be used mainly for comparisons with singletons such as None (if x is None:). Using is to compare numbers or strings is unreliable, because results depend on Python's internal caching of small integers and strings.

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a
print(a == b)    # True  – same contents
print(a is b)    # False – different objects
print(a is c)    # True  – c refers to the same object as a
```

2.11 Comparing Floating-Point Values

Since the Grade Calculator and Calculator read float values, it is important to know that floating-point numbers are stored in binary with limited precision. For example, 0.1 + 0.2 == 0.3 evaluates to False because the sum is actually 0.30000000000000004. When comparing computed decimal values for equality, use math.isclose(x, y) rather than ==. Comparisons with >= and < against thresholds, as in the grade ladder, are generally safe.

2.12 Common Errors

- Using the assignment operator = instead of the comparison operator == in a condition results in a SyntaxError.
- Forgetting the colon at the end of if, elif or else, or misaligning the indentation of blocks.
- Writing elif conditions in the wrong order so that an earlier, broader condition hides a later one.
- Not validating input: float(input()) raises ValueError when the user types non-numeric text (handled later in Experiment 7).

Key Points: if is one-way, if-else two-way, and if-elif-else multi-way selection; only the first True branch of a ladder executes. else is optional. The ternary form is x if condition else y. == compares values while is compares object identity.

Programs

Program 1: Grade Calculator

```
marks = float(input("Enter marks (0-100): "))
if marks >= 90:
    grade = "O (Outstanding)"
elif marks >= 75:
    grade = "A (Excellent)"
elif marks >= 60:
    grade = "B (Good)"
elif marks >= 50:
    grade = "C (Average)"
elif marks >= 40:
    grade = "D (Pass)"
else:
    grade = "F (Fail)"
print(f"Grade: {grade}")
```

Program 2: Simple Calculator

```
a = float(input("Enter first number: "))
op = input("Enter operator (+, -, *, /): ")
b = float(input("Enter second number: "))
if op == "+":
    print("Result:", a + b)
elif op == "-":
    print("Result:", a - b)
elif op == "*":
    print("Result:", a * b)
elif op == "/":

    if b != 0:
        print("Result:", a / b)
    else:
        print("Error: Division by zero!")
else:
    print("Invalid operator")
```

Practice Questions:

1. Check whether the year is a leap year or not.
2. Check whether the number is positive, negative or zero
3. Program to print Largest of three numbers

Viva Questions

1. What is the purpose of elif?
2. Can if work without else?
3. What is a nested if?
4. What is a ternary operator in Python?
5. Differentiate == and is operator.

Assessment	
Timely Submission (5)	
Code Correctness (5)	
Viva Voce (10)	
Total Marks (20)	

Experiment 3

Experiment No.	3
Title	Design and Implementation of User-Defined Functions and Lambda Expressions
Aim	To design and implement user-defined functions including recursive functions and lambda expressions in Python.
Objectives	Define and call functions, use parameters and return values, implement recursion, use lambda and higher-order functions.
Date	
Marks	
Faculty Signature	

3. Theory

3.1 Concept of a Function

A function is a named, self-contained block of code that performs a specific task and can be executed (called) whenever needed. Functions are the primary tool for modular programming: a large problem is divided into smaller sub-problems, each solved by a separate function. The main advantages of functions are:

- **Reusability** – code is written once and called many times, avoiding duplication.
- **Modularity and readability** – each function has a single, clearly named responsibility.
- **Abstraction** – the caller needs to know *what* a function does, not *how* it does it.
- **Easier testing and debugging** – each function can be tested independently.

Python functions are of four kinds: built-in functions (print(), len(), max()), functions from modules (math.sqrt(), random.randint()), user-defined functions created with def, and anonymous functions created with lambda.

3.2 Defining and Calling a Function

A function is defined using the def keyword followed by the function name, a pair of parentheses containing optional parameters, and a colon. The indented body may begin with a docstring – a triple-quoted string describing the function, which can be viewed with help(function_name) or function_name.__doc__. Defining a function only creates it; the body executes only when the function is called by writing its name followed by parentheses and arguments.

```
def area_of_rectangle(length, width):     # definition (parameters)
    """Return the area of a rectangle."""
    return length * width

a = area_of_rectangle(5, 3)               # call (arguments)
print(a)                                  # 15
```

Parameters are the variable names listed in the function definition; arguments are the actual values passed when the function is called. In Python, functions are first-class objects: they can be assigned to variables, stored in lists, passed as arguments to other functions and returned from functions. This property is the basis of lambdas and higher-order functions.

3.3 Types of Arguments

Type	Description	Example
Positional	Values are matched to parameters by their position	power(2, 3)
Keyword	Values are matched by parameter name; order does not matter	power(exp=3, base=2)
Default	A parameter has a default value used when no argument is supplied	def power(base, exp=2):
Variable-length *args	Collects any number of extra positional arguments into a **tuple**	def total(*nums):
Variable-length **kwargs	Collects any number of extra keyword arguments into a **dictionary**	def info(**data):

```
def power(base, exp=2):           # default argument
    return base ** exp
print(power(5))          # 25  (exp takes default value 2)
print(power(2, 5))       # 32  (positional)
print(power(exp=3, base=4))   # 64  (keyword)

def total(*nums):                 # nums is a tuple
    return sum(nums)
print(total(1, 2, 3, 4))          # 10
```

```
def student(**details):          # details is a dict
    for key, value in details.items():
        print(key, ":", value)
student(name="Asha", roll=101, branch="CSE")
```

When different kinds of parameters are combined, they must appear in the order: ordinary positional parameters (those with default values after those without), *args, keyword-only parameters, and finally **kwargs. A common pitfall is using a mutable default value such as def add(item, lst=[]): the default list is created only once, when the function is defined, and is shared by all calls. The safe pattern is lst=None followed by if lst is None: lst = [] inside the body.

3.4 The return Statement

The return statement ends the execution of a function and sends a value back to the caller. A function without a return statement (or with a bare return) returns the special value None. A function can return several values separated by commas; Python packs them into a tuple, which the caller can unpack.

```
def min_max(numbers):
    return min(numbers), max(numbers)    # returns a tuple


low, high = min_max([4, 9, 1, 7])
print(low, high)                    # 1 9
```

3.5 Scope and Lifetime of Variables

The scope of a variable is the region of the program in which the variable can be accessed. A variable created inside a function is local: it exists only while the function executes (its lifetime) and cannot be accessed outside. A variable created at the top level of a program is global. When a name is used, Python searches for it in four scopes in a fixed order known as the LEGB rule:

Scope	Meaning
L – Local	Names assigned inside the current function
E – Enclosing	Names in the local scope of any enclosing (outer) function, for nested functions
G – Global	Names assigned at the top level of the module (file)
B – Built-in	Names predefined by Python, such as print, len, range

To modify a global variable inside a function, it must be declared with the global keyword; to modify a variable of an enclosing function, the nonlocal keyword is used. Without these declarations, an assignment creates a new local variable.

```
count = 0               # global variable
def increment():
    global count
    count += 1          # modifies the global variable
increment(); increment()
print(count)            # 2
```

3.6 Recursion

Recursion is a technique in which a function calls itself to solve a smaller instance of the same problem. It follows the divide-and-conquer principle. Every correct recursive function has two parts:

- **Base case** – the simplest instance, solved directly without a further recursive call. It stops the recursion. For factorial, 0! = 1! = 1.
- **Recursive case** – the problem is expressed in terms of a smaller version of itself, moving towards the base case. For factorial, n! = n × (n − 1)!.

Each call is stored on the call stack as a separate frame holding its own local variables. The calls first build up (winding) until the base case is reached, and then the results are multiplied as the calls return (unwinding). The evaluation of factorial(4) is:

```
factorial(4)
= 4 * factorial(3)
= 4 * (3 * factorial(2))
= 4 * (3 * (2 * factorial(1)))     # base case reached: returns 1
= 4 * (3 * (2 * 1))
= 24
```

If the base case is missing or never reached, the function keeps calling itself until Python's recursion limit (about 1000 frames by default, see sys.getrecursionlimit()) is exceeded and a RecursionError is raised. Recursion is natural for problems with a recursive structure – factorial, Fibonacci, Tower of Hanoi, tree traversal and binary search – but it uses more memory than an equivalent loop.

Aspect	Recursion	Iteration
Mechanism	Function calls itself	Loop repeats a block

Aspect	Recursion	Iteration
Termination	Base case	Loop condition becomes False
Memory	One stack frame per call (higher)	Constant (lower)
Code	Short and elegant for recursive problems	Usually longer but faster
Failure mode	RecursionError (stack overflow)	Infinite loop

3.7 Lambda (Anonymous) Functions

A lambda function is a small, anonymous function defined in a single line using the lambda keyword. Its syntax is lambda parameters: expression. It can take any number of arguments but contains only one expression, whose value is returned automatically – no return keyword is written. Lambda functions cannot contain statements such as loops or assignments. They are mainly used for short, throw-away functions passed as arguments to other functions.

```
square = lambda x: x ** 2
print(square(6))                    # 36

add = lambda a, b: a + b
print(add(3, 4))                    # 7

students = [("Ravi", 72), ("Asha", 91), ("Kiran", 85)]
students.sort(key=lambda s: s[1], reverse=True)   # sort by marks
print(students)   # [('Asha', 91), ('Kiran', 85), ('Ravi', 72)]
```

Feature	Normal Function (def)	Lambda Function
Name	Has a name	Anonymous (may be assigned to a variable)
Body	Any number of statements	A single expression
Return	Explicit return statement	Value of the expression returned implicitly
Docstring	Supported	Not supported
Typical use	Reusable, complex logic	Short functions passed to map(), filter(), sorted()

3.8 Higher-Order Functions: map(), filter() and reduce()

A higher-order function is a function that accepts another function as an argument or returns a function. Python's three classic higher-order functions support the functional programming style used in Program 2:

- map(function, iterable) – applies the function to **every** item and returns an iterator of the results. map(lambda x: x**2, nums) produces the squares.
- filter(function, iterable) – keeps only those items for which the function returns True. filter(lambda x: x % 2 == 0, nums) keeps the even numbers.
- reduce(function, iterable) – from the functools module; repeatedly applies a two-argument function to accumulate the items into a single value.

In Python 3, map() and filter() return lazy iterators rather than lists, which is why the program wraps them in list(). For the list 1 to 10, reduce(lambda x, y: x * y, nums) is evaluated step by step as ((((1 × 2) × 3) × 4) … × 10) = 3628800, which is 10!. The same results can often be written with list comprehensions, e.g. [x**2 for x in nums] and [x for x in nums if x % 2 == 0].

Key Points: Functions are defined with def and executed only when called. *args collects extra positional arguments into a tuple and **kwargs collects keyword arguments into a dictionary. Name lookup follows the LEGB rule. Every recursive function needs a base case. A lambda is an anonymous single-expression function commonly used with map(), filter(), reduce() and sorted().

Programs

Program 1: Factorial using Recursion

```
def factorial(n):
    """Returns factorial of n recursively"""
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

num = int(input("Enter a number: "))
print(f"Factorial of {num} = {factorial(num)}")
```

Program 2: Lambda with map, filter, reduce

```
from functools import reduce

nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Square each number
squares = list(map(lambda x: x**2, nums))
print("Squares:", squares)

# Filter even numbers
evens = list(filter(lambda x: x % 2 == 0, nums))
print("Evens:", evens)

# Product of all numbers
product = reduce(lambda x, y: x * y, nums)
print("Product:", product)
```

Practice Questions:

1. Write a function to calculate the area of a triangle given its base and height.
2. Write a recursive function to find the factorial of a number.
3. Write a function that returns two values swapped.
4. Write a function to count the number of words in a sentence.

Viva Questions

1. What is a function in Python?
2. Explain the difference between a function and a lambda.
3. What is recursion? Give an example.
4. What are *args and **kwargs?
5. What is scope in Python?

Assessment	
Timely Submission (5)	
Code Correctness (5)	
Viva Voce (10)	
Total Marks (20)	

Experiment 4

Experiment No.	4
Title	Implementation of Python Data Structures Using Lists, Tuples, Sets, and Dictionaries
Aim	To implement and apply Python built-in data structures to solve computational problems.
Objectives	Create and manipulate lists, tuples, sets, and dictionaries; apply built-in methods; choose appropriate structures.
Date	
Marks	
Faculty Signature	

4. Theory

4.1 Introduction to Data Structures

A data structure is a way of organizing and storing data in memory so that it can be accessed and modified efficiently. Choosing the right data structure directly affects the simplicity and speed of a program. Python provides four powerful built-in collection types – list, tuple, set and dictionary – which can hold multiple values under a single name. They differ in three important properties:

- **Ordering** – whether elements keep a defined position and can be accessed by an index.
- **Mutability** – whether the collection can be changed (items added, removed or modified) after creation.
- **Duplicates** – whether the same value may appear more than once.

Lists and tuples are sequence types (ordered and indexed), sets are unordered collections of unique elements, and dictionaries are mapping types that associate keys with values.

4.2 Lists

A list is an ordered, mutable collection of items enclosed in square brackets [ ] and separated by commas. A list can contain elements of different types and can contain duplicates. Internally, a Python list is a dynamic array of references: it grows automatically as elements are added, and any element can be accessed directly by its index in constant time.

Indexing and Slicing

Each element has a positive index starting from 0 at the left and a negative index starting from −1 at the right. Slicing with list[start:stop:step] extracts a sub-list; as with range(), the stop index is excluded. Accessing an index beyond the list raises IndexError, but slicing beyond the limits simply returns the available elements.

```
marks = [85, 72, 91, 64, 78]
print(marks[0], marks[-1])     # 85 78
print(marks[1:4])              # [72, 91, 64]
print(marks[::-1])             # [78, 64, 91, 72, 85]  (reversed copy)
marks[1] = 75                  # lists are mutable
print(len(marks), 91 in marks) # 5 True
```

Common List Methods

Method	Description	Example (lst = [3, 1, 2])
append(x)	Adds x at the end	lst.append(4) → [3, 1, 2, 4]
extend(iterable)	Adds all items of another iterable	lst.extend([5, 6])
insert(i, x)	Inserts x at index i	lst.insert(0, 9) → [9, 3, 1, 2]
remove(x)	Removes the first occurrence of x	lst.remove(1) → [3, 2]
pop([i])	Removes and returns item at i (last by default)	lst.pop() → returns 2
sort()	Sorts the list in place	lst.sort() → [1, 2, 3]
reverse()	Reverses the list in place	lst.reverse() → [2, 1, 3]
index(x) / count(x)	Position of x / number of occurrences of x	lst.index(2) → 2

Note that sort() modifies the list and returns None, whereas the built-in sorted(lst) returns a new sorted list and leaves the original unchanged.

4.3 Tuples

A tuple is an ordered, immutable collection of items enclosed in parentheses ( ). Once created, its elements cannot be changed, added or removed; attempting t[0] = 5 raises TypeError. Tuples support indexing, slicing, len(), membership testing and only two methods: count() and index().

A tuple with a single element must include a trailing comma: (5,) is a tuple, while (5) is just the integer 5. Writing values separated by commas creates a tuple automatically (packing), and assigning a tuple to several variables extracts its elements (unpacking).

```
point = (3, 4)
single = (5,)
record = 101, "Asha", 91       # packing
roll, name, marks = record     # unpacking
print(name, type(single))      # Asha <class 'tuple'>
```

Tuples are preferred when data should not change (e.g. coordinates, days of the week, database records). Because they are immutable and fixed in size, they generally use less memory than equivalent lists and can be created slightly faster. Most importantly, tuples containing only immutable elements are hashable, so they can be used as dictionary keys or set elements, which lists cannot.

4.4 Sets

A **set** is an unordered, mutable collection of unique, hashable elements, written in curly braces { }. Duplicates are removed automatically, so converting a list to a set is the simplest way to eliminate repeated values. Because sets are unordered, they do not support indexing or slicing. An empty set must be created with set(), because {} creates an empty dictionary.

Sets are implemented as hash tables, so testing membership (x in s) takes constant time on average, much faster than searching a list. Elements are added with add() and removed with remove() (raises KeyError if absent) or discard() (no error). An immutable version, frozenset, can itself be used as a dictionary key. The mathematical set operations used in Program 2 are summarized below for A = {1, 2, 3, 4, 5} and B = {4, 5, 6, 7, 8}.

Operation	Operator	Method	Result
Union – elements in A or B	A | B	A.union(B)	{1, 2, 3, 4, 5, 6, 7, 8}
Intersection – elements in both	A & B	A.intersection(B)	{4, 5}
Difference – in A but not in B	A - B	A.difference(B)	{1, 2, 3}
Symmetric difference – in exactly one	A ^ B	A.symmetric_difference(B)	{1, 2, 3, 6, 7, 8}
Subset – every element of A is in B	A <= B	A.issubset(B)	False

```
nums = [1, 2, 2, 3, 3, 3]
unique = set(nums)
print(unique)          # {1, 2, 3}
empty = set()          # NOT {}
```

4.5 Dictionaries

A dictionary is a mutable collection of key–value pairs, written as {key: value}. It maps unique keys to values, so a value is retrieved by its key rather than by a numeric position – much like looking up a word in a real dictionary. Keys must be unique and hashable (strings, numbers or tuples of immutables); values can be of any type, including lists or other dictionaries. Assigning to an existing key overwrites its old value. Since Python 3.7, dictionaries preserve insertion order, although they are still accessed by key and not by index.

Accessing a missing key with d[key] raises KeyError; the method d.get(key, default) returns a default value instead. This is used in Experiment 5 for word counting: freq.get(w, 0) + 1.

Method / Operation	Description
d[key] = value	Adds a new pair or updates the value of an existing key
d.get(key, default)	Returns the value, or default if the key is absent
d.keys() / d.values() / d.items()	Views of keys, values and (key, value) pairs
d.update(other)	Adds or overwrites pairs from another dictionary
d.pop(key)	Removes the key and returns its value
del d[key]	Deletes the pair with the given key
key in d	Tests whether a key exists (average constant time)

Program 1 stores student records as a nested dictionary: the outer dictionary maps each roll number to an inner dictionary holding that student's name and marks. The loop for roll, info in students.items() unpacks each key–value pair, and info["marks"] accesses the inner value.

```
students = {101: {"name": "Alice", "marks": 85}}
students[102] = {"name": "Bob", "marks": 72}     # add record
print(students[101]["name"])                     # Alice
for roll, info in students.items():
    print(roll, info["name"], info["marks"])
```

The display function uses f-string format specifiers: {roll:<8} prints the value left-aligned (<) in a field 8 characters wide, producing neat columns. Similarly, > right-aligns, ^ centres and .2f shows two decimal places.

4.6 Hashing and Collision Handling

Dictionaries and sets are implemented as hash tables. When a key is stored, Python computes an integer hash value using hash(key) and uses it to decide the slot in an internal array where the entry is placed. Lookup repeats the same computation, so it goes directly to the right slot instead of scanning all entries – this is why dictionary access takes constant time on average. This is also why keys must be immutable: if a key could change, its hash would change and the entry could no longer be found.

A collision occurs when two different keys map to the same slot. CPython resolves collisions using open addressing: it probes a sequence of other slots, determined by a perturbation of the hash value, until it finds an empty slot (for insertion) or the matching key (for lookup). Matching is confirmed by comparing both the hash and the key with ==. If two keys are genuinely equal, it is not a collision – the new value simply replaces the old one. The table is automatically resized when it becomes too full, keeping collisions rare.

4.7 Mutable and Immutable Types

An object is mutable if its contents can be changed in place without creating a new object, and immutable if any "change" actually creates a new object. The distinction matters because assignment in Python copies references, not objects.

Immutable Types	Mutable Types
int, float, complex, bool	list
str	dict
tuple	set
frozenset, bytes	bytearray, user-defined objects (by default)

```
a = [1, 2, 3]
b = a          # b refers to the SAME list (aliasing)
b.append(4)
print(a)       # [1, 2, 3, 4]  – a is also changed

c = a.copy()   # shallow copy: a new list
c.append(5)
```

```
print(a)        # [1, 2, 3, 4]  – a is unaffected


s = "hi"
s = s + "!"      # a new string object is created
```

A shallow copy (copy(), list(a) or a[:]) creates a new outer container but shares nested objects; copy.deepcopy() creates fully independent copies of nested structures such as the student records dictionary.

4.8 Comprehensions

Comprehensions provide a compact, readable syntax for building a new collection from an existing iterable, optionally with a condition. They are generally faster than equivalent loops using append().

```
nums = [1, 2, 3, 4, 5, 6]
squares  = [x * x for x in nums]                # list comprehension
evens    = {x for x in nums if x % 2 == 0}      # set comprehension
sq_dict  = {x: x * x for x in nums}             # dictionary comprehension
result   = {r: ("Pass" if m >= 40 else "Fail")
           for r, m in {101: 85, 104: 32}.items()}
print(sq_dict)   # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36}
print(result)    # {101: 'Pass', 104: 'Fail'}
```

4.9 Comparison and Selection of Data Structures

Property	List	Tuple	Set	Dictionary
Syntax	[1, 2]	(1, 2)	{1, 2}	{"a": 1}
Ordered	Yes	Yes	No	Insertion order (3.7+)
Mutable	Yes	No	Yes	Yes
Duplicates	Allowed	Allowed	Not allowed	Keys unique; values may repeat
Access	By index	By index	Membership only	By key
Empty form	[]	()	set()	{}
Typical use	Changing ordered data	Fixed records, dict keys	Unique items, set algebra	Fast lookup by key

As a guideline: use a list when order matters and items change; a tuple when the data is fixed; a set when uniqueness or fast membership tests are needed; and a dictionary when each value is naturally identified by a key, such as a roll number.

Key Points: Lists are ordered and mutable; tuples are ordered and immutable; sets are unordered collections of unique elements; dictionaries map unique hashable keys to values. Dictionaries and sets are hash tables, giving average constant-time lookup. Assignment copies references, so modifying a mutable object through one name affects all names bound to it.

Programs

Program 1: Student Record System using Dictionary

```
students = {}

def add_student(roll, name, marks):
    students[roll] = {"name": name, "marks": marks}

def display_all():
    print(f"{"Roll":<8} {"Name":<15} {"Marks":<8} {"Grade"}")
    for roll, info in students.items():
        m = info["marks"]
        g = "Pass" if m >= 40 else "Fail"
        print(f"{roll:<8} {info['name']:<15} {m:<8} {g}")

add_student(101, "Alice", 85)
add_student(102, "Bob", 72)
add_student(103, "Charlie", 91)
display_all()
```

Program 2: Set Operations

```
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

print("Union:", A | B)
print("Intersection:", A & B)
print("Difference (A-B):", A - B)
```

```
print("Symmetric Difference:", A ^ B)
print("Is A subset of B?", A.issubset(B))
```

Practice Questions:

1. Find the maximum and minimum value in a tuple
2. Accept 5 numbers from the user and store them in a list.
3. Write a function to count the number of words in a sentence.
4. Write a lambda function to compute simple interest.

Viva Questions

1. Difference between list and tuple?
2. What is a set? Can it have duplicates?
3. What is dictionary comprehension?
4. How does Python handle key collisions in a dict?
5. What are mutable and immutable types?

Assessment	
Timely Submission (5)	
Code Correctness (5)	
Viva Voce (10)	
Total Marks (20)	

Experiment 5

Experiment No.	5
Title	Pattern Matching and Validation Using Regular Expressions in Python
Aim	To implement pattern matching and text validation using the re module in Python.
Objectives	Use Python re module, write regex patterns, validate email/phone/date, use match/search/findall/sub.
Date	
Marks	
Faculty Signature	

5. Theory

5.1 Introduction to Regular Expressions

A regular expression (regex or regexp) is a sequence of characters that defines a search pattern for text. Instead of searching for one fixed string, a regex describes a whole family of strings – for example, "one or more digits", "a word that starts with a capital letter", or "a valid e-mail address". Regular expressions are used for four main tasks: validation (checking that input follows a required format), searching, extraction of matching parts, and substitution (find-and-replace). Python supports regular expressions through the built-in re module, imported with import re.

A regex engine reads the pattern and the target string and tries to match the pattern starting at successive positions. Ordinary characters such as letters and digits match themselves; metacharacters have special meanings and give regular expressions their power.

5.2 Raw Strings

Regex patterns contain many backslashes (\d, \w, \b). In a normal Python string, the backslash is also an escape character – for instance, "\b" is interpreted as the backspace character. To avoid this double interpretation, patterns are written as raw strings by prefixing them with r, as in r"\b[a-zA-Z]+\b". In a raw string, backslashes are passed unchanged to the regex engine.

5.3 Metacharacters

Symbol	Meaning	Example	Matches
.	Any single character except newline	c.t	cat, cot, c9t
^	Start of the string	^Py	"Python" but not "CPy"
$	End of the string	on$	"Python"
*	0 or more repetitions of the preceding item	ab*	a, ab, abbb
+	1 or more repetitions	ab+	ab, abbb (not a)
?	0 or 1 repetition (optional)	colou?r	color, colour
{m} / {m,n}	Exactly m / between m and n repetitions	\d{3}	123, 987
[ ]	Character class: any one character listed	[aeiou]	any vowel
[^]	Negated class: any character not listed	[^0-9]	any non-digit
|	Alternation (OR)	cat|dog	cat or dog
( )	Grouping and capturing	(ab)+	ab, abab
\	Escapes a metacharacter or starts a special sequence	\.	a literal dot

5.4 Special Sequences

Sequence	Meaning	Equivalent Class
\d	Any decimal digit	[0-9]
\D	Any non-digit character	[^0-9]
\w	Any word character (letter, digit, underscore)	[a-zA-Z0-9_]
\W	Any non-word character	[^a-zA-Z0-9_]
\s	Any whitespace (space, tab, newline)	[\t\n\r\f\v]
\S	Any non-whitespace character	[^ \t\n\r\f\v]

Sequence	Meaning	Equivalent Class
\b	Word boundary (between \w and \W, or string edge)	–
\B	Not a word boundary	–

For Python 3 strings, \d and \w also match Unicode digits and letters from other scripts; the equivalents above describe the ASCII case. Inside square brackets, most metacharacters lose their special meaning (e.g. [.] matches a literal dot). A hyphen between two characters denotes a range ([a-z]), so a literal hyphen is written at the start or end of the class or escaped as \-.

5.5 Quantifiers: Greedy and Lazy Matching

The quantifiers *, +, ? and {m,n} are greedy by default: they match as many characters as possible while still allowing the overall pattern to succeed. Appending ? makes them lazy (non-greedy), so they match as few characters as possible (*?, +?, ??, {m,n}?).

```
import re
html = "<b>bold</b> and <i>italic</i>"
print(re.findall(r"<.*>", html))    # ['<b>bold</b> and <i>italic</i>']  greedy
print(re.findall(r"<.*?>", html))   # ['<b>', '</b>', '<i>', '</i>']       lazy
```

5.6 Groups and Alternation

Parentheses group part of a pattern so that a quantifier applies to the whole group, and they capture the matched text for later use. In the phone pattern (\+91|0)?, the group contains two alternatives – the country code "+91" or a leading "0" – and the ? makes the whole group optional. Captured groups are retrieved from a match object with group(n) or groups(). Named groups use the syntax (?P<name>...). A non-capturing group (?:...) groups without capturing. Note that if a pattern contains capturing groups, re.findall() returns the captured groups instead of the whole matches.

```
m = re.search(r"(\d{2})-(\d{2})-(\d{4})", "DOB: 15-08-2005")
print(m.group())       # 15-08-2005  (whole match)
print(m.groups())      # ('15', '08', '2005')
day, month, year = m.groups()
```

5.7 Functions of the re Module

Function	Description	Return Value
re.match(p, s)	Tries to match the pattern only at the beginning of the string	Match object or None

Function	Description	Return Value
re.search(p, s)	Scans the entire string for the first location that matches	Match object or None
re.fullmatch(p, s)	Succeeds only if the whole string matches the pattern	Match object or None
re.findall(p, s)	Finds all non-overlapping matches	List of strings (or tuples of groups)
re.finditer(p, s)	Like findall, but lazily yields match objects	Iterator of match objects
re.sub(p, repl, s)	Replaces every match with repl	New string
re.split(p, s)	Splits the string at every match	List of strings
re.compile(p)	Compiles a pattern into a reusable regex object	Pattern object

A match object is truthy, so it can be used directly in an if statement; bool(re.match(...)) in Program 1 converts it to True/False. Its methods include group() (matched text), start() and end() (positions) and span() (a tuple of both).

5.8 re.match() versus re.search()

The key difference is where matching is attempted. re.match() checks only at position 0, whereas re.search() checks every position until a match is found.

```
text = "I love Python"
print(re.match(r"Python", text))    # None – not at the beginning
print(re.search(r"Python", text))   # <re.Match object; span=(7, 13), match='Python'>
print(re.match(r"I", text))         # match at position 0
```

For validation, the pattern must describe the entire input. This is achieved either with re.match() plus the anchors ^ and $ (as in Program 1) or, more directly, with re.fullmatch(). Without the $ anchor, re.match(r"\d{10}", "98765432101234") would still succeed because the first ten characters match.

5.9 Flags and Compiled Patterns

- re.IGNORECASE (re.I) – case-insensitive matching.
- re.MULTILINE (re.M) – ^ and $ match at the start and end of every line.
- re.DOTALL (re.S) – . also matches newline characters.
- re.VERBOSE (re.X) – allows whitespace and comments inside the pattern for readability.

When the same pattern is used repeatedly, compiling it once with re.compile() makes the code cleaner and avoids re-parsing: email_re = re.compile(pattern) followed by email_re.match(email).

5.10 Analysis of the Patterns Used in the Programs

a) E-mail Pattern: ^[a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}$

Part	Meaning
^	Match must start at the beginning of the string
[a-zA-Z0-9._%+\-]+	User name: one or more letters, digits or the symbols . _ % + -
@	Exactly one literal @ symbol
[a-zA-Z0-9.\-]+	Domain name: one or more letters, digits, dots or hyphens
\.	A literal dot before the top-level domain
[a-zA-Z]{2,}	Top-level domain of at least two letters (com, org, in)
$	Match must end at the end of the string

b) Indian Mobile Number Pattern: ^(\+91|0)?[6-9]\d{9}$

Part	Meaning
^	Start of string
(\+91|0)?	Optional prefix: "+91" (the + is escaped) or "0"
[6-9]	First digit of the mobile number must be 6, 7, 8 or 9
\d{9}	Exactly nine more digits (ten digits in total)
$	End of string – no extra characters permitted

Accordingly, "+919876543210" and "9876543210" are valid, while "1234567890" is invalid because it starts with 1. Such patterns are practical approximations; the complete e-mail standard (RFC 5322) is far more complex, so production systems also confirm an address by sending a verification e-mail.

c) Word Extraction and Frequency

In Program 2, \b[a-zA-Z]+\b matches whole words made only of letters; the word boundaries ensure that punctuation such as "." and "!" is excluded. re.findall() returns all words as a list, and a dictionary with get() counts their frequencies after converting them to lowercase. re.sub(r"Python", "Java", text) replaces every occurrence and returns a new string, since strings are immutable.

5.11 Python Modules for Text Processing

Besides re, Python's standard library offers several modules for handling text: string (constants such as string.ascii_letters, string.punctuation), textwrap (wrapping and formatting paragraphs), difflib (comparing sequences of text), unicodedata (Unicode character properties) and collections.Counter, which counts word frequencies in a single line: Counter(words).most_common(3). Many simple tasks – str.startswith(), str.isdigit(), str.split(), str.replace() – do not require regular expressions at all and are faster and more readable when a fixed string is involved.

Key Points: A regex describes a pattern of text; patterns are written as raw strings. match() checks only the beginning, search() scans the whole string, fullmatch() requires the entire string to match, findall() returns all matches and sub() replaces them. \d, \w and \s match digits, word characters and whitespace. Quantifiers are greedy unless followed by ?. Validation patterns must be anchored with ^ and $.

Programs

Program 1: Email and Phone Validator

```
import re

def validate_email(email):
    pattern = r"^[a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}$"
    return bool(re.match(pattern, email))

def validate_phone(phone):
    pattern = r"^(\+91|0)?[6-9]\d{9}$"
    return bool(re.match(pattern, phone))

emails = ["test@example.com", "invalid-email", "user@domain.org"]
phones = ["+919876543210", "1234567890", "9876543210"]

for e in emails:
    print(f"{e}: {"Valid" if validate_email(e) else "Invalid"}")
for p in phones:
    print(f"{p}: {"Valid" if validate_phone(p) else "Invalid"}")
```

Program 2: Extract and Count Words

```
import re

text = "Python is great. Python is easy. I love Python!"

# Extract all words
words = re.findall(r"\b[a-zA-Z]+\b", text)
print("Words:", words)

# Count word frequency
freq = {}
for w in words:
    freq[w.lower()] = freq.get(w.lower(), 0) + 1
print("Frequency:", freq)

# Replace word
new_text = re.sub(r"Python", "Java", text)
print("After replace:", new_text)
```

Practice Questions:

1. Write a Python program using regular expressions to validate a password according to the following conditions:
    - Minimum 8 characters
    - At least one uppercase letter
    - At least one lowercase letter
    - At least one digit
    - At least one special character

Viva Questions

1. What is a regular expression?
2. Difference between re.match and re.search?
3. What does \d mean in regex?
4. What is a greedy quantifier?
5. Name two Python modules that support text processing.

Assessment	
Timely Submission (5)	
Code Correctness (5)	
Viva Voce (10)	
Total Marks (20)	

Experiment 6

Experiment No.	6
Title	Design and Implementation of Object-Oriented Programming Concepts in Python
Aim	To design and implement OOP concepts including classes, objects, inheritance, polymorphism, and encapsulation.
Objectives	Define classes and create objects, implement inheritance, apply polymorphism and encapsulation, use constructors and destructors.
Date	
Marks	
Faculty Signature	

6. Theory

6.1 Programming Paradigms: Procedural versus Object-Oriented

In procedural programming, a program is organized as a sequence of functions that operate on data stored separately, often in global variables. As programs grow, it becomes difficult to track which function changes which data. Object-Oriented Programming (OOP) organizes software around objects – units that bundle together data (attributes) and the operations (methods) that act on that data. OOP models real-world entities such as a bank account, a student or a shape, and it promotes reusability, maintainability and security of data. Python is a multi-paradigm language, and in Python everything – integers, strings, lists and functions – is an object.

Aspect	Procedural Programming	Object-Oriented Programming
Basic unit	Function	Object (instance of a class)
Data and code	Kept separate	Bundled together
Data security	Data often globally accessible	Data hidden through encapsulation
Reusability	Through function calls	Through inheritance and composition

Aspect	Procedural Programming	Object-Oriented Programming
Suitable for	Small, linear tasks	Large, complex, evolving systems

6.2 Classes and Objects

A class is a user-defined blueprint or template that describes the attributes and behaviour common to all objects of a particular kind. It is defined using the class keyword. An object (or instance) is a concrete entity created from a class; each object has its own copy of the instance attributes. Creating an object is called instantiation and is done by calling the class like a function: acc = BankAccount("Alice", 5000). Many objects can be created from one class, just as many houses can be built from a single architectural plan.

The first parameter of every instance method is conventionally named self. It refers to the particular object on which the method is called. When acc.deposit(2000) is executed, Python internally calls BankAccount.deposit(acc, 2000), so self receives acc.

Instance Attributes and Class Attributes

Instance attributes are defined with self.name = value, usually inside the constructor, and belong to one object. Class attributes are defined directly inside the class body and are shared by all objects of the class – for example, a common bank name or an interest rate, or a counter of the number of objects created.

```
class Student:
    college = "DYPIU"              # class attribute (shared)
    count = 0

    def __init__(self, name, roll):
        self.name = name           # instance attributes
        self.roll = roll
        Student.count += 1

s1 = Student("Asha", 101)
s2 = Student("Ravi", 102)
print(s1.name, s2.name, Student.college, Student.count)
# Asha Ravi DYPIU 2
```

6.3 Constructors and Destructors

A constructor is a special method that is called automatically when an object is created. In Python, the constructor is __init__(self, ...). Its job is to initialize the new object's attributes, possibly using default parameter values such as balance=0 in the BankAccount class. (Strictly, the object is created by __new__ and then initialized by __init__.) A class can have only one __init__; different ways of creating an object are supported through default arguments.

A destructor, __del__(self), is called when an object is about to be destroyed – in CPython, when its reference count drops to zero, e.g. after del obj removes the last reference. Because Python manages memory automatically through garbage collection, the exact time of destruction is not guaranteed, and destructors are rarely needed; resources such as files are better released using with statements.

```
class Demo:
    def __init__(self, name):
        self.name = name
        print(self.name, "created")
    def __del__(self):
        print(self.name, "destroyed")


d = Demo("Object-1")   # Object-1 created
del d                  # Object-1 destroyed
```

6.4 Types of Methods

Method Type	Decorator	First Parameter	Accesses	Use
Instance method	none	self	Instance and class data	Operations on one object (e.g. deposit)
Class method	@classmethod	cls	Class data only	Factory methods, class-level counters
Static method	@staticmethod	none	Neither	Utility functions logically related to the class

6.5 Encapsulation and Data Hiding

Encapsulation is the bundling of data and methods into a single unit (the class) together with restricting direct access to some of the object's data. The internal state of the object can then be changed only

through controlled methods that enforce rules. In the BankAccount program, the balance cannot be set to an arbitrary value from outside; it changes only through deposit() and withdraw(), which check that the amount is positive and that sufficient funds exist.

Python does not have access-specifier keywords like private in Java or C++. Instead, it uses naming conventions:

Naming	Access Level	Meaning
balance	Public	Freely accessible from anywhere
_balance	Protected (by convention)	Signals "internal use"; still accessible, but should not be used outside the class and its subclasses
__balance	Private (name mangling)	Python renames it to _BankAccount__balance, so acc.__balance raises AttributeError

The double underscore prefix triggers name mangling, which makes accidental access from outside the class difficult. It is not absolute security – acc._BankAccount__balance still works – but it clearly expresses the programmer's intention. Controlled read access is provided through getter methods such as get_balance(), or more elegantly through the @property decorator.

```
acc = BankAccount("Alice", 5000)
print(acc.get_balance())       # 5000 – allowed via method
print(acc.__balance)           # AttributeError: no attribute '__balance'


class Account:
    def __init__(self, bal):
        self.__bal = bal
    @property
    def balance(self):         # read-only property
        return self.__bal
print(Account(900).balance)    # 900 – accessed like an attribute
```

6.6 Inheritance

Inheritance allows a new class (the child, derived or sub class) to acquire the attributes and methods of an existing class (the parent, base or super class). The child class can use the inherited members as they are, add new members, or override inherited methods. Inheritance represents an "is-a" relationship

– a Circle *is a* Shape – and promotes code reuse. It is written as class Child(Parent):. Every Python class implicitly inherits from the built-in class object.

Type	Structure	Example
Single	One child inherits from one parent	Circle → Shape
Multilevel	A chain of inheritance	Puppy → Dog → Animal
Hierarchical	Several children share one parent	Circle, Rectangle, Triangle → Shape
Multiple	One child inherits from two or more parents	class C(A, B):
Hybrid	Combination of two or more of the above	Hierarchical + multiple

Program 2 uses hierarchical inheritance: Circle, Rectangle and Triangle all inherit describe() from Shape and each provides its own area().

The super() Function

super() returns a proxy object that delegates method calls to the parent class. It is most commonly used in a child's constructor to reuse the parent's initialization: super().__init__(...). Calling the parent by name, as in Parent.__init__(self, ...), also works, but it hard-codes the parent name and requires passing self explicitly. super() follows the Method Resolution Order (MRO) – the order in which Python searches classes for a method, computed by the C3 linearization algorithm and viewable with ClassName.mro() – so it works correctly with multiple inheritance, where naming the parent directly can cause a method to be called twice.

```
class Person:
    def __init__(self, name):
        self.name = name


class Student(Person):
    def __init__(self, name, roll):
        super().__init__(name)          # reuse parent initialization
        self.roll = roll

s = Student("Asha", 101)
```

```
print(s.name, s.roll)                # Asha 101
print([c.__name__ for c in Student.mro()])   # ['Student', 'Person', 'object']
```

6.7 Polymorphism

Polymorphism ("many forms") means that the same interface – the same method name or operator – can behave differently depending on the object it is applied to. In Program 2, the loop calls s.describe() on each shape, and describe() in turn calls self.area(). At run time Python calls the area() belonging to the actual class of each object, so a single piece of code computes the areas of circles, rectangles and triangles. Python supports polymorphism in several ways:

- **Method overriding** – a child class redefines a method inherited from the parent with the same name and parameters. Circle.area() overrides Shape.area().
- **Duck typing** – "if it walks like a duck and quacks like a duck, it is a duck". Python does not check an object's type; any object that provides the required method can be used. len() works on strings, lists and dictionaries alike.
- **Operator overloading** – operators are given meaning for user-defined classes by defining special methods, e.g. __add__ for +. The + operator already behaves polymorphically for numbers (addition), strings (concatenation) and lists (joining).

Python does not support traditional method overloading (several methods with the same name but different parameter lists in one class); a later definition simply replaces the earlier one. The same effect is achieved with default arguments or *args.

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, other):            # overloads +
        return Vector(self.x + other.x, self.y + other.y)
    def __str__(self):                   # used by print()
        return f"({self.x}, {self.y})"

print(Vector(1, 2) + Vector(3, 4))       # (4, 6)
```

6.8 Abstraction

Abstraction means exposing only the essential features of an object while hiding the implementation details. A user of the Shape classes only needs to know that every shape has an area() method, not the formula used inside it. The base class Shape in Program 2 achieves informal abstraction by raising NotImplementedError, which forces subclasses to implement area(). Python's abc module provides

formal abstract base classes: a class that inherits from ABC and contains a method decorated with @abstractmethod cannot be instantiated, and any subclass that fails to implement the abstract method also cannot be instantiated.

```
from abc import ABC, abstractmethod
import math

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return math.pi * self.r ** 2

# Shape()  -> TypeError: Can't instantiate abstract class Shape
print(round(Circle(7).area(), 2))      # 153.94
```

6.9 Special (Magic / Dunder) Methods

Methods whose names begin and end with double underscores are called special or dunder methods. Python calls them automatically in response to built-in operations, allowing user-defined objects to behave like built-in types. The expression self.__class__.__name__ used in describe() reads the name of the object's actual class, so it prints "Circle", "Rectangle" or "Triangle".

Method	Invoked By	Purpose
__init__	ClassName(...)	Initialize a new object
__str__	print(obj), str(obj)	Readable, user-friendly representation
__repr__	repr(obj), interactive shell	Unambiguous developer representation
__len__	len(obj)	Return the size of the object
__eq__, __lt__	==, <	Comparison of objects
__add__	obj1 + obj2	Operator overloading for +
__del__	Object destruction	Clean-up (destructor)

6.10 Summary of the Four Pillars of OOP

Pillar	Definition	Illustration in This Experiment
Encapsulation	Bundling data with methods and restricting direct access	Private __balance, changed only via deposit() / withdraw()
Inheritance	Deriving new classes from existing ones to reuse code	Circle, Rectangle and Triangle inherit from Shape
Polymorphism	One interface, many implementations	Each subclass overrides area(); describe() works for all
Abstraction	Showing essential features, hiding details	Shape defines area() without implementing it

Key Points: A class is a blueprint and an object is its instance. __init__ is the constructor and self refers to the current object. Double-underscore attributes are name-mangled to discourage outside access. super() calls parent methods according to the MRO. Method overriding and duck typing provide run-time polymorphism; abstract base classes enforce abstraction.

Programs

Program 1: Bank Account with Encapsulation

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.__owner = owner
        self.__balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print(f"Deposited ₹{amount}. Balance: ₹{self.__balance}")

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
            print(f"Withdrawn ₹{amount}. Balance: ₹{self.__balance}")
        else:
            print("Insufficient funds!")
```

```
    def get_balance(self):
        return self.__balance

acc = BankAccount("Alice", 5000)
acc.deposit(2000)
acc.withdraw(1500)
print("Final Balance:", acc.get_balance())
```

Program 2: Inheritance and Polymorphism – Shape Area Calculator

```
import math

class Shape:
    def area(self):
        raise NotImplementedError("Subclass must implement area()")

    def describe(self):
        print(f"{self.__class__.__name__}: Area = {self.area():.2f}")

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return math.pi * self.r ** 2

class Rectangle(Shape):
    def __init__(self, l, w): self.l, self.w = l, w
    def area(self): return self.l * self.w

class Triangle(Shape):
    def __init__(self, b, h): self.b, self.h = b, h
    def area(self): return 0.5 * self.b * self.h

shapes = [Circle(7), Rectangle(5, 10), Triangle(6, 8)]
for s in shapes:
    s.describe()
```

Practice Questions:

1. Create a class Student with the following attributes: name, roll_no, marks. Create an object and display the student details
2. Create a class Student with the following attributes: name, roll_no, marks.
   Create methods to:

   - Display student details
   - Calculate average marks
   - Display Pass/Fail

   Consider the student as **Pass if average marks ≥ 40**.

Viva Questions

1. What is a class and an object?
2. Explain the four pillars of OOP.
3. What is the purpose of __init__?
4. What is method overriding?
5. What is the difference between super() and parent class name?

Assessment	
Timely Submission (5)	
Code Correctness (5)	
Viva Voce (10)	
Total Marks (20)	

Experiment 7

Experiment No.	7
Title	Implementation of File Handling and Exception Handling Mechanisms in Python
Aim	To implement file operations and handle runtime exceptions gracefully using Python.
Objectives	Perform read/write/append on files, handle exceptions using try-except-finally, create custom exceptions.
Date	
Marks	
Faculty Signature	

7. Theory

Part A – File Handling

7.1 Need for File Handling

Variables, lists and dictionaries are stored in main memory (RAM), which is volatile: all data is lost when the program ends or the computer is switched off. A file is a named location on secondary storage (hard disk, SSD, pen drive) that stores data persistently. File handling enables programs to save results, read input data, maintain records and exchange data with other programs. Files are of two types:

- Text files – store data as human-readable characters encoded using an encoding such as UTF-8, organized into lines separated by the newline character \n (e.g. .txt, .csv, .py).
- Binary files – store data as raw bytes in the same form as it exists in memory; they are not human-readable (e.g. images, audio, .exe, .pdf).

Working with a file always involves three steps: open the file, perform read/write operations, and close the file.

7.2 Opening a File: open() and File Modes

The built-in function open(file, mode="r", encoding=None) opens a file and returns a file object (also called a file handle) through which all operations are performed. The file name may be a relative path

(relative to the current working directory) or an absolute path. The mode determines what operations are permitted.

Mode	Description	If File Exists	If File Does Not Exist
"r"	Read only (default)	Opens; pointer at start	FileNotFoundError
"w"	Write only	Erases existing content	Creates a new file
"a"	Append – write at end	Keeps content; pointer at end	Creates a new file
"x"	Exclusive creation	FileExistsError	Creates a new file
"r+"	Read and write	Opens; pointer at start	FileNotFoundError
"w+"	Write and read	Erases content	Creates a new file
"a+"	Append and read	Keeps content	Creates a new file
"rb", "wb", "ab"	Same as r, w, a in binary mode	As above	As above

7.3 Reading from a File

Method	Description	Return Type
read()	Reads the entire file	One string
read(n)	Reads at most n characters from the current position	String
readline()	Reads one line, including the trailing \n; returns "" at end of file	String
readlines()	Reads all remaining lines	List of strings
for line in f:	Iterates line by line without loading the whole file	One string per iteration

Iterating directly over the file object, as done in Program 1, is the most memory-efficient method because only one line is kept in memory at a time; this makes it suitable even for very large files. Each line ends with a newline character, so line.strip() removes it (together with surrounding spaces) before split(",") breaks the line into fields. Values read from a text file are always strings; int(marks) is needed for arithmetic.

7.4 Writing to a File

The method write(string) writes a string to the file and returns the number of characters written. Unlike print(), it does not add a newline automatically, which is why Program 1 writes "101,Alice,85\n". The method writelines(list_of_strings) writes several strings in one call, again without adding newlines. Only strings can be written to a text file, so numbers must be converted with str() or formatted with an f-string. Opening an existing file in "w" mode destroys its previous contents; "a" mode should be used to add new records to the end.

```
with open("log.txt", "a") as f:
    f.write("New entry\n")
    f.writelines(["line A\n", "line B\n"])
    print("Printed into file", file=f)     # print() can also write to a file
```

7.5 The File Pointer: tell() and seek()

Every open file maintains a file pointer (cursor) indicating the position where the next read or write will occur. Reading or writing advances the pointer automatically. f.tell() returns the current position, and f.seek(offset, whence) moves the pointer to a new position, where whence is 0 (from the beginning, default), 1 (from the current position) or 2 (from the end). In text mode, only seeking relative to the beginning (or to the end with offset 0) is fully supported.

7.6 Closing Files and the with Statement

Data written to a file is first stored in a memory buffer and is physically written to disk when the buffer is full, when flush() is called, or when the file is closed. Calling f.close() flushes the buffer and releases the operating-system resource. If a program forgets to close a file, or an exception occurs before close() is reached, data may be lost.

The with statement solves this problem. It uses the file object as a context manager: the file is opened at the start of the block and is closed automatically when the block ends, even if an exception occurs inside it. This is the recommended way to work with files and is used in both parts of Program 1.

```
# Without with – must close manually
f = open("students.txt", "r")
data = f.read()
f.close()


# With with – closed automatically
with open("students.txt", "r") as f:
```

```
    data = f.read()
print(f.closed)          # True
```

7.7 Text Mode ("r") versus Binary Mode ("rb")

Aspect	Text Mode ("r")	Binary Mode ("rb")
Data returned	str objects	bytes objects
Encoding	Bytes decoded using an encoding (e.g. UTF-8)	No decoding; raw bytes
Newlines	Platform line endings (e.g. \r\n) translated to \n	Left unchanged
Used for	Text, CSV, source code, logs	Images, audio, video, PDFs, copying any file

Because the default encoding depends on the operating system, it is good practice to specify it explicitly: open("data.txt", "r", encoding="utf-8"). For structured comma-separated data, Python's csv module (csv.reader, csv.writer) correctly handles quoted fields that themselves contain commas, which a simple split(",") cannot.

7.8 Working with the File System

The os and pathlib modules provide functions for managing files and folders: os.path.exists(path) checks existence, os.remove(path) deletes a file, os.rename(old, new) renames it, os.listdir() lists a directory and os.getcwd() returns the current working directory. The modern object-oriented alternative is pathlib.Path, e.g. Path("students.txt").exists() and Path("students.txt").read_text().

Part B – Exception Handling

7.9 Types of Errors

- **Syntax errors** – violations of Python's grammar, such as a missing colon or unmatched bracket. They are detected before the program runs, and the program does not start.
- **Runtime errors (exceptions)** – errors that occur while a syntactically correct program is executing, such as dividing by zero, opening a missing file or converting "abc" to an integer.
- **Logical errors** – the program runs without crashing but produces wrong results, e.g. using the wrong formula. Python cannot detect these; they are found by testing.

An exception is an object that represents a runtime error. When an error occurs, Python raises an exception. If it is not handled, the program terminates abruptly and prints a traceback – a report

showing the sequence of function calls that led to the error and the exception type and message. Exception handling allows the program to catch the exception, respond sensibly (display a message, retry, use a default value) and continue running.

7.10 Common Built-in Exceptions

Exception	Raised When	Example
ZeroDivisionError	Division or modulo by zero	10 / 0
ValueError	Correct type but inappropriate value	int("abc")
TypeError	Operation applied to an inappropriate type	"5" + 3
NameError	An undefined variable is used	print(xyz)
IndexError	Sequence index out of range	[1, 2][5]
KeyError	Dictionary key not found	{"a": 1}["b"]
AttributeError	Object has no such attribute	acc.__balance
FileNotFoundError	File to be read does not exist	open("none.txt")
PermissionError	Insufficient access rights to a file	Writing to a protected file
ModuleNotFoundError	Imported module cannot be found	import numpyy

Exception Hierarchy

All exceptions are classes arranged in a hierarchy. The root is BaseException; its subclass Exception is the base of all ordinary errors. KeyboardInterrupt (Ctrl + C) and SystemExit inherit directly from BaseException, not from Exception. Therefore except Exception: catches all normal errors while still allowing the user to interrupt the program, whereas a bare except: catches everything and should be avoided. Related exceptions share parents – for example, FileNotFoundError and PermissionError are both subclasses of OSError, and ZeroDivisionError is a subclass of ArithmeticError.

7.11 The try, except, else and finally Blocks

- **try** – contains the code that might raise an exception. As soon as an exception occurs, the remaining statements in the try block are skipped.
- **except** – handles the exception. except ValueError as e: catches a specific type and binds the exception object to e, whose message is displayed with print(e). Several except blocks can

follow one try block, and one block can handle multiple types written as a tuple: except (ValueError, TypeError):.

- **else** – optional; runs only if the try block completed without any exception. It keeps the "success" code separate from the risky code.
- **finally** – optional; runs always, whether or not an exception occurred and even if it was not handled or a return was executed. It is used for clean-up such as closing files or releasing connections.

Situation	try	except	else	finally
No exception occurs	Completes	Skipped	Runs	Runs
Exception occurs and is handled	Stops at the error	Matching block runs	Skipped	Runs
Exception occurs, no matching except	Stops at the error	None run	Skipped	Runs, then the exception propagates

Python checks the except blocks from top to bottom and executes the **first** one whose type matches (including parent classes). Therefore specific exceptions must be written **before** general ones. In Program 2, InvalidAgeError and ValueError are listed before the catch-all Exception; if except Exception were written first, it would catch everything and the specific handlers would never run.

```
try:
    a = int(input("Numerator: "))
    b = int(input("Denominator: "))
    result = a / b
except ZeroDivisionError:
    print("Cannot divide by zero")
except ValueError as e:
    print("Invalid input:", e)
else:
    print("Result =", result)        # only if no exception
finally:
    print("Done")                    # always
```

7.12 Raising Exceptions

The raise statement triggers an exception deliberately when the program detects an invalid situation that cannot be handled at that point, e.g. raise ValueError("Age must be 18 or above."). This transfers responsibility for handling the error to the caller. Inside an except block, a bare raise re-raises the current exception after, for example, logging it; raise NewError(...) from e raises a new exception while preserving the original cause.

7.13 User-Defined (Custom) Exceptions

Built-in exceptions describe general errors. For problems specific to an application – an invalid age, insufficient balance, an out-of-range mark – programmers define custom exceptions by creating a class that inherits from Exception (or one of its subclasses). The class can be empty (pass), as InvalidAgeError is in Program 2, or it can store extra information. Custom exceptions make error handling more meaningful, because the caller can catch exactly the condition it is interested in.

```
class InsufficientFundsError(Exception):
    def __init__(self, balance, amount):
        super().__init__(f'Balance {balance} is less than {amount}")
        self.balance, self.amount = balance, amount


try:
    raise InsufficientFundsError(500, 800)
except InsufficientFundsError as e:
    print("Error:", e)      # Error: Balance 500 is less than 800
```

In Program 2, entering a negative age raises InvalidAgeError; entering 15 raises ValueError from verify_age(); and entering text such as "abc" makes int() itself raise a ValueError, which is caught by the same handler and prints Python's message "invalid literal for int() with base 10: 'abc'". In every case, the finally block prints "Verification process complete."

7.14 Propagation of Unhandled Exceptions

When an exception is raised inside a function and is not handled there, the function stops immediately and the exception is passed to its caller. This propagation continues up the call stack until a matching except block is found. If no handler exists at any level, Python terminates the program and prints the traceback. This mechanism allows low-level functions to report errors while higher-level code decides how to respond.

7.15 Combining File and Exception Handling

File operations are a common source of exceptions, so they are frequently combined with exception handling to produce robust programs:

```
filename = input("Enter file name: ")
try:
    with open(filename, "r", encoding="utf-8") as f:
        for line in f:
            roll, name, marks = line.strip().split(",")
            print(roll, name, int(marks))
except FileNotFoundError:
    print(f"'{filename}' does not exist.")
except ValueError:
    print("File contains an incorrectly formatted record.")
finally:
    print("File processing finished.")
```

Key Points: "w" mode overwrites, "a" appends and "r" raises FileNotFoundError if the file is missing. The with statement closes files automatically. Text mode returns str, binary mode returns bytes. else runs only when no exception occurs; finally always runs. Specific except blocks must precede general ones. Custom exceptions inherit from Exception, and unhandled exceptions propagate up the call stack and terminate the program.

Programs

Program 1: Student Records File I/O

```
# Writing to file
with open("students.txt", "w") as f:
    f.write("101,Alice,85\n")
    f.write("102,Bob,72\n")
    f.write("103,Charlie,91\n")
print("Records written.")

# Reading from file
print("\nStudent Records:")
with open("students.txt", "r") as f:
    for line in f:
```

```
    roll, name, marks = line.strip().split(",")
    print(f"Roll: {roll}, Name: {name}, Marks: {marks}")
```

Program 2: Exception Handling with Custom Exception

```
class InvalidAgeError(Exception):
    """Custom exception for invalid age"""
    pass

def verify_age(age):
    if age < 0:
        raise InvalidAgeError("Age cannot be negative.")
    elif age < 18:
        raise ValueError("Age must be 18 or above.")
    return True

try:
    age = int(input("Enter age: "))
    if verify_age(age):
        print("Access granted!")
except InvalidAgeError as e:
    print(f"Custom Error: {e}")
except ValueError as e:
    print(f"Value Error: {e}")
except Exception as e:
    print(f"Unexpected Error: {e}")
finally:
    print("Verification process complete.")
```

Practice Questions

1. Write a Python program to read a text file and display:
    - Number of lines
    - Number of words
    - Number of characters

2. Create a file containing student records. Ask the user whether they want to add another student. If yes, accept: Roll No, Name, Marks and append the information to the existing file. Finally, display all records.

Viva Questions

1. What is file handling?
2. Difference between "r" and "rb" mode?
3. What is the purpose of finally?
4. How do you create a custom exception?
5. What happens if an exception is not handled?

Assessment	
Timely Submission (5)	
Code Correctness (5)	
Viva Voce (10)	
Total Marks (20)	

Experiment 8

Experiment No.	8
Title	Data Analysis and Visualization Using NumPy, Pandas, and Matplotlib Libraries
Aim	To perform data analysis using NumPy and Pandas, and visualize results using Matplotlib.
Objectives	Use NumPy arrays for numerical operations, load and analyze data with Pandas, create charts using Matplotlib.
Date	
Marks	
Faculty Signature	

8. Theory

8.1 Data Analysis and the Python Ecosystem

Data analysis is the process of inspecting, cleaning, transforming and modelling data to discover useful information and support decisions. A typical workflow consists of: (1) collecting or loading data, (2) cleaning it (handling missing or incorrect values), (3) exploring and analysing it using statistics, (4) visualizing the results, and (5) interpreting and communicating conclusions. Python has become the leading language for this work because of three foundational libraries:

- **NumPy** (Numerical Python) – fast multi-dimensional arrays and mathematical operations; the foundation of scientific computing in Python.
- **Pandas** – high-level, labelled data structures (Series and DataFrame) for loading, cleaning and analysing tabular data; built on top of NumPy.
- **Matplotlib** – a comprehensive library for creating static, animated and interactive visualizations.

These are third-party libraries, installed with pip install numpy pandas matplotlib (they are pre-installed in distributions such as Anaconda and in Google Colab), and imported using the standard aliases np, pd and plt.

8.2 NumPy and the ndarray

The core of NumPy is the ndarray (N-dimensional array): a grid of values that are all of the same data type, stored in a single contiguous block of memory. This design makes NumPy arrays much faster and more memory-efficient than Python lists for numerical work. A Python list stores references to separate objects scattered in memory, and each element must be type-checked during every operation. A NumPy array stores raw numbers side by side and performs operations in optimized, pre-compiled C code.

Feature	Python List	NumPy Array
Element types	Can be mixed	Homogeneous (single dtype)
Memory layout	References to scattered objects	Contiguous block of raw values
Speed for numeric operations	Slow (interpreted loop)	Fast (vectorized C code)
* 2 operation	Repeats the list: [1, 2, 1, 2]	Multiplies each element: [2, 4]
Mathematical functions	Require loops or the math module	Built in: mean, std, dot, …
Size	Can grow and shrink (append)	Fixed at creation

Creating Arrays and Array Attributes

```
import numpy as np
a = np.array([85, 72, 91, 64])          # from a list
z = np.zeros(5)                         # [0. 0. 0. 0. 0.]
r = np.arange(0, 10, 2)                 # [0 2 4 6 8]
l = np.linspace(0, 1, 5)                # [0. 0.25 0.5 0.75 1.]
m = np.arange(6).reshape(2, 3)          # 2 rows x 3 columns
print(m.ndim, m.shape, m.size, m.dtype) # 2 (2, 3) 6 int64
```

The attribute ndim gives the number of dimensions, shape the size along each dimension, size the total number of elements, and dtype the element type (the default integer type may be int32 on some Windows systems).

Vectorization, Broadcasting and Boolean Indexing

Vectorization means applying an operation to an entire array at once instead of writing an explicit loop: scores + 5 adds 5 to every element. Broadcasting is the set of rules that allows arrays of different

but compatible shapes to be combined, for example adding a scalar or a one-dimensional row to every row of a matrix. Boolean indexing selects elements using a condition: the comparison produces an array of True/False values that acts as a mask.

```
scores = np.array([85, 72, 91, 64, 78])
print(scores + 5)            # [90 77 96 69 83]  – vectorized
print(scores > 80)           # [ True False  True False False]
print(scores[scores > 80])   # [85 91]          – boolean indexing
print(np.argmax(scores))     # 2  – index of the maximum value
```

8.3 Statistical Measures Using NumPy

Program 1 computes descriptive statistics, which summarize a dataset using measures of central tendency (mean, median) and dispersion (variance, standard deviation, range, percentiles). For *n* values $x_1, x_2, \ldots, x_n$:

- **Mean** $\mu = (\Sigma x_i) / n$ – the arithmetic average. It is sensitive to extreme values (outliers).
- **Median** – the middle value of the sorted data; for an even count, the average of the two middle values. It is robust to outliers.
- **Variance** $\sigma^2 = \Sigma (x_i - \mu)^2 / n$ – the average squared deviation from the mean.
- **Standard deviation** $\sigma = \sqrt{\sigma^2}$ – the typical distance of values from the mean, expressed in the same units as the data.
- **Percentile** – the value below which a given percentage of the data lies; the 75th percentile is also called the third quartile (Q3). NumPy uses linear interpolation between data points by default.

For the scores used in Program 1 – [85, 72, 91, 64, 78, 88, 95, 70, 55, 82] – the results are:

Measure	NumPy Function	Value	Working
Mean	np.mean()	78.0	780 / 10
Median	np.median()	80.0	Sorted middle values 78 and 82 → (78 + 82) / 2
Variance	np.var()	144.8	$\Sigma(x - 78)^2 / 10 = 1448 / 10$
Standard deviation	np.std()	≈ 12.03	$\sqrt{144.8}$
Minimum / Maximum	np.min() / np.max()	55 / 95	Range = 40
75th percentile	np.percentile(s, 75)	87.25	Interpolated between 85 and 88

Important: np.std() and np.var() compute the population measures (dividing by n, ddof=0) by default, whereas the Pandas methods .std() and .var() compute the sample measures (dividing by n − 1, ddof=1). For the same data, Pandas reports a standard deviation of about 12.68 instead of 12.03. Use np.std(s, ddof=1) to obtain the sample value in NumPy.

8.4 Pandas: Series and DataFrame

Pandas provides two primary data structures. A Series is a one-dimensional labelled array: a column of values together with an index of labels. A DataFrame is a two-dimensional labelled table of rows and columns, similar to a spreadsheet or an SQL table; each column is a Series, and different columns may have different data types. In Program 2, the DataFrame is created from a dictionary in which each key becomes a column name and each list becomes the column's values; Pandas assigns the default row index 0 to 4.

```
import pandas as pd
s = pd.Series([85, 72, 91], index=["Alice", "Bob", "Charlie"])
print(s["Bob"])                    # 72


df = pd.DataFrame({"Name": ["Alice", "Bob"], "Math": [85, 72]})
```

Loading, Saving and Inspecting Data

Function / Attribute	Purpose
pd.read_csv("file.csv")	Load a CSV file into a DataFrame (also read_excel, read_json, read_sql)
df.to_csv("out.csv", index=False)	Save a DataFrame to a CSV file
df.head(n) / df.tail(n)	First / last n rows (default 5)
df.shape	Tuple of (number of rows, number of columns)
df.info()	Column names, data types and non-null counts
df.describe()	Count, mean, std, min, quartiles and max of numeric columns
df.columns / df.dtypes	Column labels / data type of each column

Selecting and Filtering Data

A single column is selected with df["Math"] (returns a Series) and several columns with a list, df[["Math", "Science"]] (returns a DataFrame). df.loc[row_label, column_label] selects by labels,

while df.iloc[row_position, column_position] selects by integer positions. Rows are filtered with Boolean conditions, combined using & (and) and | (or) with each condition in parentheses.

```
print(df.loc[0, "Name"])                    # label-based
print(df.iloc[0, 1])                        # position-based
toppers = df[df["Math"] > 80]               # filter rows
both = df[(df["Math"] > 70) & (df["Science"] > 70)]
ranked = df.sort_values("Math", ascending=False) # sort rows
```

Creating Columns and the axis Parameter

New columns are created by assignment, as in df["Total"] = ... The axis parameter tells Pandas the direction of an operation: axis=0 (the default) operates down the rows and produces one result per column, while axis=1 operates across the columns and produces one result per row. Thus df[["Math","Science","English"]].sum(axis=1) gives each student's total, whereas .mean() with the default axis gives the class average of each subject (Math 78.0, Science 79.2, English 78.6).

Handling Missing Data

Real datasets often contain missing values, represented in Pandas as NaN. They are detected with df.isnull().sum(), removed with df.dropna(), or replaced with df.fillna(value) – for example, filling missing marks with the column mean. Most statistical methods skip NaN values automatically.

Grouping Data with groupby()

The groupby() method implements the split–apply–combine strategy: the data is split into groups based on the values of one or more columns, a function (such as mean, sum, count or max) is applied to each group independently, and the results are combined into a new table. It is used for questions such as "average marks per division" or "total sales per region".

```
data = pd.DataFrame({
   "Division": ["A", "B", "A", "B", "A"],
   "Marks":    [85,  72,  91,  64,  78]})
print(data.groupby("Division")["Marks"].mean())
# Division
# A    84.666667
# B    68.000000
```

8.5 Data Visualization with Matplotlib

Data visualization is the graphical representation of data. Charts reveal trends, comparisons, distributions and outliers that are difficult to see in tables of numbers. Matplotlib's pyplot module

provides a simple interface for creating plots. Every plot consists of a Figure – the overall window or page – containing one or more Axes, each of which is an individual plot with its own x-axis, y-axis, title, labels and legend.

In Program 2, fig, axes = plt.subplots(1, 2, figsize=(12, 5)) creates a figure 12 × 5 inches in size with one row and two columns of Axes; axes[0] receives the bar chart and axes[1] the pie chart. Pandas DataFrames have a built-in plot() method that uses Matplotlib internally, and the ax= argument directs the plot to a particular Axes.

Plot Type	Function	Best Used For
Line chart	plt.plot(x, y)	Trends over time or ordered values
Bar chart	plt.bar(x, h)	Comparing quantities across categories
Histogram	plt.hist(data, bins=10)	Distribution (frequency) of a numeric variable
Scatter plot	plt.scatter(x, y)	Relationship or correlation between two variables
Pie chart	plt.pie(values, labels=...)	Parts of a whole (proportions)
Box plot	plt.boxplot(data)	Median, quartiles, spread and outliers

Customizing a Plot

```
import matplotlib.pyplot as plt
subjects = ["Math", "Science", "English"]
avg = [78.0, 79.2, 78.6]
plt.bar(subjects, avg, color="steelblue", label="Class average")
plt.title("Average Marks by Subject")
plt.xlabel("Subject")
plt.ylabel("Marks")
plt.ylim(0, 100)
plt.legend()
plt.grid(axis="y", linestyle="--")
plt.savefig("average.png", dpi=150)    # save BEFORE show()
plt.show()
```

Titles and axis labels are set with set_title(), set_xlabel() and set_ylabel() when working with Axes objects, or with plt.title(), plt.xlabel() and plt.ylabel() in the simple pyplot style. plt.tight_layout()

automatically adjusts spacing so that titles and labels of neighbouring subplots do not overlap. In the pie chart, autopct="%1.1f%%" displays each slice's percentage with one decimal place; %% prints a literal percent sign. plt.savefig() must be called before plt.show(), because in many environments the figure is cleared after it is displayed, and the saved image would then be blank.

8.6 Integration of the Three Libraries

The three libraries work together as a single toolkit. Pandas stores each DataFrame column internally as a NumPy array, so NumPy functions can be applied directly to Pandas data (np.mean(df["Math"])), and df.to_numpy() converts a DataFrame to an array. Pandas' plot() method, in turn, draws its charts with Matplotlib. A typical analysis therefore loads and cleans data with Pandas, performs numerical computation with NumPy, and presents the results with Matplotlib.

Key Points: NumPy arrays are homogeneous, contiguous and vectorized, making them much faster than lists for numerical work. np.std() uses the population formula by default, Pandas .std() the sample formula. A DataFrame is a labelled 2-D table; loc selects by label and iloc by position; axis=1 works row-wise. groupby() follows split–apply–combine. Choose the chart type according to the question: bar for comparison, line for trends, histogram for distribution, scatter for relationships and pie for proportions.

Programs

Program 1: NumPy Statistical Analysis

```
import numpy as np

scores = np.array([85, 72, 91, 64, 78, 88, 95, 70, 55, 82])

print("Scores:", scores)
print("Mean:", np.mean(scores))
print("Median:", np.median(scores))
print("Std Dev:", np.std(scores))
print("Min:", np.min(scores), " Max:", np.max(scores))
print("Variance:", np.var(scores))

# Sort and percentile
print("Sorted:", np.sort(scores))
print("75th Percentile:", np.percentile(scores, 75))
```

Program 2: Pandas Data Analysis and Matplotlib Visualization

```
import pandas as pd
import matplotlib.pyplot as plt

# Create DataFrame
data = {
    "Name": ["Alice","Bob","Charlie","Diana","Eve"],
    "Math": [85, 72, 91, 64, 78],
    "Science": [88, 65, 82, 90, 71],
    "English": [76, 80, 68, 74, 95]
}
df = pd.DataFrame(data)

# Analysis
df["Total"] = df[["Math","Science","English"]].sum(axis=1)
df["Average"] = df["Total"] / 3
print(df)
print("\nClass Average:")
print(df[["Math","Science","English"]].mean())

# Visualization
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Bar chart
df.plot(x="Name", y=["Math","Science","English"],
        kind="bar", ax=axes[0], title="Subject-wise Marks")
axes[0].set_ylabel("Marks")

# Pie chart – average marks distribution
avg = df[["Math","Science","English"]].mean()
axes[1].pie(avg, labels=avg.index, autopct="%1.1f%%")
axes[1].set_title("Average Marks Distribution")

plt.tight_layout()
```

```
plt.savefig("analysis.png")
plt.show()
```

Practice Questions

1. Create a NumPy array containing marks of 10 students.

   Calculate and display:

   - Mean
   - Median
   - Maximum
   - Minimum
   - Standard deviation
   - Variance
   - Sorted marks

2. Create a dataset containing the marks of five students in: Math, Science, English. Generate the following charts:

   - **Bar chart** – student-wise marks
   - **Line chart** – marks comparison
   - **Pie chart** – subject-wise average marks

   Add: Chart title, X-axis label, Y-axis label, Legend where appropriate
   Save the chart as an image file.

Viva Questions

1. What is NumPy? How is it different from a Python list?
2. What is a DataFrame in Pandas?
3. Name 3 types of plots in Matplotlib.
4. What is the use of groupby() in Pandas?
5. How do you read a CSV file in Pandas?

Assessment	
Timely Submission (5)	
Code Correctness (5)	
Viva Voce (10)	
Total Marks (20)	